SHORT ANSWER QUESTIONS

Question 1

Differences between TensorFlow and PyTorch

Feature	TensorFlow	PyTorch
Origin	Developed by Google Brain	Developed by Facebook AI Research
Programming Style	Graph-based (static computation graph)	Eager execution (dynamic computation graph)
Ease of Use	More verbose and complex	More pythonic and intuitive
Debugging	Harder especially with static graphs	Easier. It integrates well with Python debugging tools.
Deployment	TensorFlow Serving, TensorFlow Lite and TensorFlow.js make deployment easy across platforms	TensorFlow is more mature in deployment
Mobile & Embedded	Strong support via TensorFlow Lite	Limited support
Visualization	TensorBoard is built-in and powerful	TensorBoard is supported, but not native. Third party tools can also be used
Community and Ecosystem	Larger corporate/ enterprise adoption	More academic and research-focused initially
Performance	Optimized for large-scale production	Excellent performance but sometimes lags slightly behind in mobile or embedded

PyTorch should be chosen when:

- Doing research and experimentation
- Writing code that feels like regular python
- There is a need for flexibility in dynamic neural networks
- You want more transparent model building and easier debugging

TensorFlow is chosen when:

- Targeting production or deployment
- Working in a large-scale organization that already uses TensorFlow ecosystem tools
- You want more tools out-of-the-box, for example, TensorFlow Lite
- There is a need to scale to distributed systems

Question 2

Use Cases for Jupyter Notebooks in AI Development

1. Exploratory Data Analysis (EDA) AND Model Prototyping
    - Before training any AI model, you need to understand the data – what features are present, what is missing, and how variables relate.
    - Datasets can be loaded on Jupyter, cleaned, and manipulated using pandas.
    - Jupyter can be used to visualize patterns with libraries like matplotlib, seaborn, or plotly.
    - Jupyter can be used to test different ML models and parameters in real-time with instant feedback.
    - An example is a data scientist working on a diabetes prediction model might explore patient health records, visualize glucose levels vs age and test a logistic regression model, all within the same notebook.
2. Training and Documenting AI/ML Models
    - You can train, evaluate, and document deep learning or ML models step by step in a single notebook.
    - Jupyter allows you to write and run training loops using frameworks like PyTorch or TensorFlow.
    - Jupyter displays real-time loss/ accuracy graphs inline.
    - Jupyter allows one to annotate every step for reproducibility or collaboration, which is ideal for students and researchers.
    - An example is a student training a convolutional neural network on the MNIST digit dataset can show model architecture, plot training/validation accuracy over epochs, and share the notebook with mentors or classmates easily.

Question 3

How space enhances NLP tasks compared to basic string operations

1. Tokenization
    - It smartly handles punctuation, contractions, and language-specific rules. Example is “don’t” becomes [“do”, “n’t”], not just one word. Basic strings, sentence.split(), breaks only by whitespace.
2. Part of Speech Tagging
    - It labels each word (token) with its grammatical role: noun, adjective, verb etc.
    - This helps in understanding structure and meaning.
    - E.g., “The cat chased the rat.” : dog = noun, chased = verb, rat = noun
3. Named Entity Recognition
    - Detects and classifies names, dates, places etc.
    - E.g., “Apple Inc. was founded in1976” : Apple Inc. = ORG, 1976 = DATE
    - This is not possible with string operations alone.
4. Dependency Parcing
    - Understands how words relate to each other (subject, object, modifiers).
    - Useful in translation, summarization or question answering
5. Lemmatization
    - It reduces words to their base form (lemma): “running”, “ran”, “runs” to “run”
    - Basic string ops can’t do this without custom rules

Question 4

Comparison between Scikit-learn and TensorFlow in terms of target applications, ease of use for beginners and Community Support

1. Target Applications

Feature	Scikit-learn	TensorFlow
Type of ML	Classical Machine Learning	Deep Learning and Neural Networks
Best For	Decision Trees, SVMs, Logistic Regression	Neural Networks, Transformers, Convolutional Neural Network
Use Case Examples	Predicting house prices, classification tasks	Image recognition, speech processing, complex deep learning
Not Ideal For	Large-scale deep learning or GPU use	Simple models like linear regression or decision trees

- Scikit-learn is best for tabular data and classical models.
- TensorFlow is best for neural networks and deep learning.

2. Ease of Use for Beginners

Aspect	Scikit-learn	TensorFlow
API Simplicity	Very intuitive; fits ML pipeline easily (fit(), predict(), etc.)	Steeper learning curve, especially for model building and training loops
Documentation	Excellent; clean and beginner-friendly	Extensive but can be overwhelming
Learning Curve	Shallow	Medium to steep (but easier with high-level APIs like Keras)
Preprocessing Tools	Built-in (StandardScaler, OneHotEncoder, etc.)	Present but more code-heavy and flexible

- Scikit-learn is easier to learn and use for small to medium-sized projects.
- TensorFlow requires more effort but is powerful once mastered, especially with tf.keras.

3. Community Support

Feature	Scikit-learn	TensorFlow
Age/ Maturity	Older and stable (started ~ 2007)	Slightly newer but fast-growing (since 2015)
Community Size	Large, especially in academia	Massive, especially in industry and deep learning
Resources	Tons of tutorials, Kaggle notebooks, and Stack Overflow answers	Extensive tutorials (including by Google), large GitHub presence, TensorBoard
Corporate Backing	Open-source (maintained by the community)	Backed by Google (strong support + enterprise tools)

- Scikit-learn has deep support in education and classic ML problems.
- TensorFlow dominates in production-grade AI and deep learning research.